

作业 7 进程终止与信号处理

同济大学计算机系操作系统 国豪 2023 拔尖班 2023

姓名 毛星博 学号 2351520

Part 1、进程终止，备考期末

1、

例题 2

已知，Unix V6++ 系统，只有这一个程序在执行。
父进程pid==2，子进程pid==3。

问，(1) 这个程序的输出是什么？
(2) 父进程有可能不睡吗？



```
#include <stdio.h>
#include <sys.h>
int main1(int argc, char* argv[])
{
    int i, j;
    if(fork())
    {
        i = wait(&j);
        printf("It is parent process. \n");           // 1
        printf("The finished child process is %d. \n", i); // 2
        printf("The exit status is %d. \n", j);         // 3
    }
    else
    {
        printf("It is child process. \n");             // 4
    }
}
```

操作系统(拔尖班 / 国豪 2024)

同济大学计算机系 邓蓉

34

(1) 父进程执行 fork() 后，子进程进入 else 块，父进程进入 if 块。父进程执行 i = wait(&j)。如果子进程还没结束，父进程会阻塞（睡觉）。子进程执行 printf("It is child process. \n");，然后自然终止（相当于 exit(0)）。子进程死后，父进程醒来，wait 返回。i 得到子进程 PID 3，j 得到状态码 0。父进程接着打印剩下的输出。因此，程序的输出为：

It is child process.

It is parent process.

The finished child process is 3.

The exit status is 0.

(2) 父进程有可能不睡。

```
if (u.u_procp->p_pid == process[i].p_ppid) /* 找所有的子进程 */
{
    hasChild = true;
    if( Process::SZOMB == process[i].p_stat) /* 回收一个终止子进程的PCB */
    { 处理终止的子进程; return; } /* PPT 21*/
}
```

因为父进程执行 fork() 之后，子进程执行 printf，父进程执行 wait 等待子进程。wait 调用时会先检查是否有已经是 SZOMB（僵尸状态）的子进程（如上图代码所示）。如果子进程运行非常快，在父进程执行到 wait(&j) 之前就已经执行完毕并调用了 exit() 变成了僵尸进程，那么父进程执行 wait 时会直接回收该僵尸进程并返回，不会进入睡眠状态。

2、



例题 4

已知， Unix V6++系统，只有这一个程序在执行。父进程 pid==2，子进程1 pid==3，子进程2 pid==4。

问， (1) 这个程序的输出是什么？
(2)子进程1 和子进程2 的PCB分别是哪个进程回收的？
(3)父进程终止后，PCB是哪个进程回收的？

```
#include<stdio.h>
#include<sys.h>
int main1(int argc,char* argv[])
{
    int i,j;
    if(fork())
    {
        printf("father.\n");
        if(fork())
        { i = wait(&j); printf("exit child = %d, exit status = %d", i, j>8) }
        else
        { printf("secondchild\n"); exit(3); }
    }
    else
    {
        sleep(2);
        printf("firstchild.\n");
        exit(5);
    }
}
```

(1) 执行顺序逻辑：

父进程先打印 father。父进程 wait。由于子进程 1 在睡觉（2 秒），子进程 2 没有睡觉，子进程 2 会先执行，打印 second child 并退出（exit code 3）。父进程的 wait 收集到子进程 2 的结束，醒来。父进程打印 exit child = 4, exit status = 3。父进程 main 结束，父进程退出。2 秒后，子进程 1 醒来，打印 first child 并退出。

因此，程序的输出为：

father.

second child.

exit child = 4, exit status = 3.

first child.

(2) 子进程 1 和子进程 2 的 PCB 分别是哪个进程回收的？

子进程 2 (PID 4)：它终止时，父进程 (PID 2) 正在 wait()。因此，由父进程 (PID 2) 回收。

子进程 1 (PID 3)：它执行时间长（sleep 2 秒）。当它醒来并终止时，父进程 (PID 2) 已经执行完并退出了。因此，子进程 1 变成了“孤儿进程”。根据 Unix 机制，孤儿进程由 1# 进程 (Init 进程) 领养并回收。

(3) 父进程终止后，PCB 是哪个进程回收的？

由题意得，此时 UnixV6++系统只有这一个程序在执行。因此可以推断，父进程 (PID2) 的父进程是 Shell 进程。所以，父进程终止后，它的 PCB 是 Shell 进程回收的。

3、假定父进程的 PID 是 007，子进程的 PID 是 008。

```
#include <stdio.h>
```

```
#include <sys.h>
```

```
main( )
```

```
{
```

```
    int i=10, j =20;
```

```
    if( i=fork( ) )
```

```
{
```

```

        printf("It is parent process. PID = %d, i = %d\n", getpid(), i);
    }
    else
    {
        sleep(100);
        printf("It is child process. PID = %d, i = %d\n", getpid(), i);
        exit(1);
    }
}

```

- 写出程序的输出。

父进程 (PID 7): fork() 返回子进程 PID 8, 赋值给 i。条件为真。

输出: It is parent process. PID = 7, i = 8

父进程没有 wait, 打印完直接就结束了。

子进程 (PID 8): fork() 返回 0, 赋值给 i。条件为假, 进 else。

睡眠 100 秒。

输出: It is child process. PID = 8, i = 0

因此, 输出为:

It is parent process. PID = 7, i = 8

It is child process. PID = 8, i = 0

- T0 时刻, 父进程创建子进程。子进程何时终止? 终止后, 子进程的 PCB 何时回收, 由谁来回收。

何时终止: $T_0 + 100 \text{ 秒} + \text{极短的执行时间 (打印语句耗时)}$ 。即大约 $T_0 + 100 \text{ 秒}$ 。

PCB 何时回收: 子进程终止的那一刻 ($T_0 + 100 \text{ 秒}$)。

由谁回收: 1 号进程 (init)。因为父进程在 T_0 时刻稍微过一点点就结束了 (因为没有 wait 调用)。子进程还要活 100 秒, 成了孤儿进程。孤儿进程的 PCB 由 1# 进程负责回收。

Part 2、信号

一、写一个 ctrl+c 杀不死的应用程序 (备考期末)

为什么 ctrl+c 可以杀死应用程序? 当按下 Ctrl+C 时, 终端驱动程序会捕获这个按键组合, 并将其转换成信号 SIGINT。在 Unix V6++ 系统中, SIGINT 的编号是 2。

每个进程在其核心数据结构 (User 结构) 中都有一个数组 u_signal[NSIG], 用来记录它对各种信号的态度。应用程序刚启动时: u_signal 数组会被清零 (0)。内核规定: 如果 u_signal[i] 的值是 0, 那么收到该信号的默认操作就是杀死进程。所以如果用户没有定义处理函数 (else 分支), 内核会执行 u.u_procp->Exit(), 直接让进程终止。

那么如何写一个 Ctrl+C 杀不死的应用程序? 只需利用 signal 系统调用捕获 SIGINT (信号值 2, 即 Ctrl+C 产生的信号), 并将其处理方式设置为自定义函数, 而不是默认的终止进程。

cantkill 应用程序如下:

```

#include <stdio.h>
#include <stdlib.h>
#include <sys.h>

void SIGINT_Handler()

```

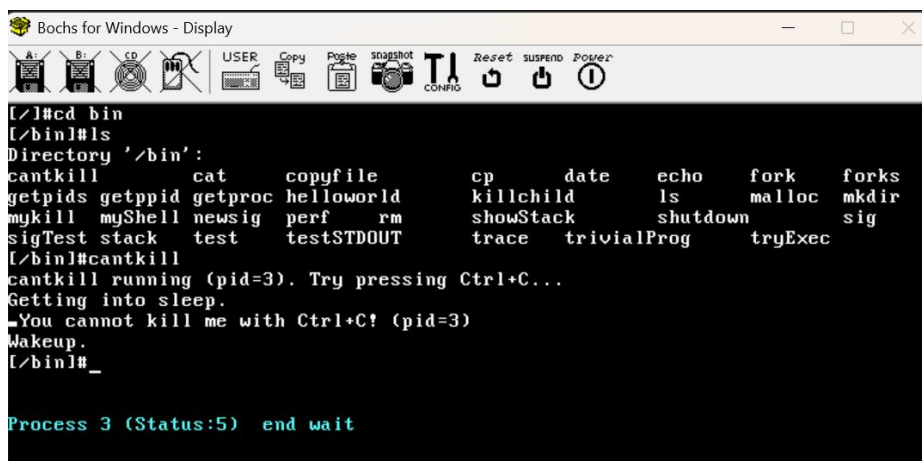
```

{
    printf("You cannot kill me with Ctrl+C! (pid=%d)\n", getpid());
}
int main1(int argc, char *argv[])
{
    /* 注册对 SIGINT(2) 的自定义处理 */
    signal(SIGINT, SIGINT_Handler);

    printf("cantkill running (pid=%d). Try pressing Ctrl+C...\n", getpid());
    printf("Getting into sleep.\n");
    sleep(50);
    printf("Wakeup.\n");
    return 1;
}

```

在 bochs 虚拟机下运行测试：运行 cantkill 之后，程序打印 Getting into sleep.之后陷入睡眠。此时按下 ctrl+c，由于注册对 SIGINT(2) 的自定义处理，程序不会被杀死，而是调用 SIGINT_Handler 打印“You cannot kill me with Ctrl+C!”，然后醒来,打印 Wakeup。注意，按下 ctrl+c 之后，Sleep 系统调用被打断，不会继续睡眠，而是直接醒来。



```

Bochs for Windows - Display
A: B: CD USER Copy Paste Snapshot T Reset SUSPEND POWER
CONFIG

[/]#cd bin
[/bin]#ls
Directory '/bin':
cantkill      cat      copyfile      cp      date      echo      fork      forks
getpids getppid getproc helloworld killchild  ls      malloc  mkdir
mykill  myShell newsig  perf      rm      showStack shutdown  sig
sigTest stack  test    testSTDOUT trace  trivialProg  tryExec

[/bin]#cantkill
cantkill running (pid=3). Try pressing Ctrl+C...
Getting into sleep.
You cannot kill me with Ctrl+C! (pid=3)
Wakeup.
[/bin]#_

Process 3 (Status:5) end wait

```

二、写一个应用程序：父进程创建一个子进程。fork 返回后，子进程无限循环。要求，父进程执行 kill 系统调用，用 SIGKILL 信号杀死子进程。

killchild 程序如下：

```

int main1(int argc, char *argv[])
{
    int pid = fork();

    if (pid == 0)
    {
        /* 子进程：打印信息后进入无限循环 */
        printf("Child (PID %d) is running...\n", getpid());
        while (1)

```

```

        sleep(1);
    }
    else if (pid > 0)
    {
        /* 父进程 */
        printf("Parent (PID %d): going to kill child (PID %d) in 1 second.\n", getpid(), pid);
        sleep(1); /* 等待 1 秒*/
        /* 发送 SIGKILL (9 号信号), 该信号不能被捕获或忽略 */
        kill(pid, SIGKILL); /* 等价于 kill(pid, 9) */
        int status;
        wait(&status); /* 回收子进程, 防止变成僵尸进程 */
        printf("Parent: Child killed. Exit status: %d\n", status);
    }
    else
    {
        /* fork 失败 */
        printf("fork() failed!\n");
    }
}
return 0;
}

```

在 bochs 虚拟机下运行测试：父进程调用 fork() 创建子进程，子进程进入无限循环。父进程等待 1 秒后，调用 kill(pid, SIGKILL) 发送 9 号信号。父进程调用 wait() 回收子进程。

```

Bochs for Windows - Display
[/]#cd bin
[/bin]#ls
Directory '/bin':
cantkill    cat        copyfile    cp         date        echo        fork        forks
getpids     getppid    getproc     helloworld killchild   ls          malloc      mkdir
mykill     myShell    newsig      perf       rm          shutdown    sig
sigTest     stack     test        testSTDOUT trace        trivialProg tryExec
[/bin]#killchild
Parent (PID 3): going to kill child (PID 4) in 1 second.
Child (PID 4) is running...
Parent: Child killed. Exit status: 1
[/bin]#_

Process 3 is exiting
end sleep
Process 3 (Status:5) end wait

```

进一步：为 Unix V6++ 系统添加应用程序 kill，这个程序有 2 个命令行参数，第 1 个参数是信号，第 2 个参数是要杀死的进程。实验设计（选做），用命令行 kill -9 pid 杀死目标进程 pid。如果想用后台作业，请参考附录文档。

mykill 应用程序设计如下：

```

int main1(int argc, char *argv[])
{
    int sig;
    int pid;

```

```

if (argc < 3)
{
    printf("Usage: kill -signal pid\n");
    exit(1);
}

/* 解析信号参数 */
sig = atoi(argv[1]);
if (sig < 0)
    sig = -sig; /* 如果用户输入 -9, 转成 9 */
/* 解析 PID 参数 */
pid = atoi(argv[2]);
if (pid <= 0)
{
    printf("Error: Invalid PID %d\n", pid);
    return 1;
}
if (sig < 1 || sig > 31)
{
    printf("Error: Invalid signal %d (must be 1-31)\n", sig);
    return 1;
}

/* 调用 kill 系统调用 */
int ret = kill(pid, sig);
if (ret < 0)
{
    printf("kill failed: no such process or permission denied\n");
    return 1;
}

printf("Sent signal %d to process %d\n", sig, pid);
return 0;
}

```

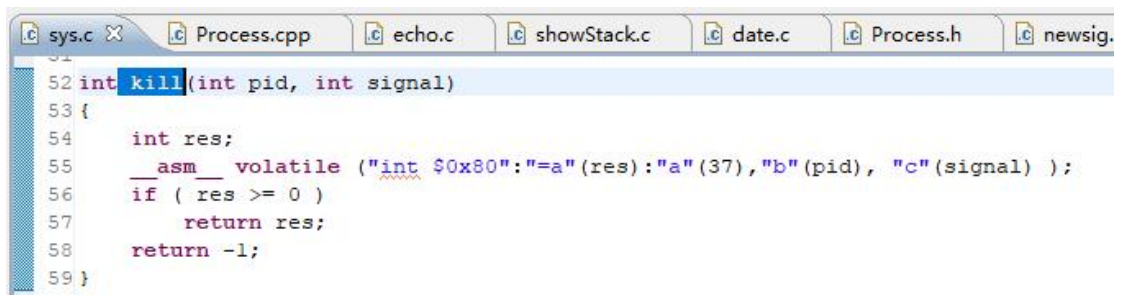
在 bochs 虚拟机下运行测试：由于 1# 进程始终运行，所以尝试杀死 1# 进程。
运行 mykill -9 1，观察到调试信息显示 Process 1 is exiting，说明 1#进程被杀死了。

```

Bochs for Windows - Display
[~/hcd bin
l/bin#ls
Directory '/bin':
cantkill  cat          copyfile      cp           date          echo          fork          forks
getpids  getppid  getproc      helloworld   killchild     ls            malloc        mkdir
mykill   myShell  newsig       perf         rm            showStack    shutdown     sig
sigTest  stack    test         testSTDOUT   trace         trivialProg   tryExec
l/bin#mykill -9 1
Sent signal 9 to process 1

Process 1 finding dead son. They are Process 3 (Status:3) wait until child process Exit! Process 0x3, p_addr 0x40F000, x_addr 0x40C000, p_size 0x5000, x_size 0x3000
Process PT:<1025,0x0> <1026,0x1> <1027,0x2> <1028,0x1> <1029,0x2> <1030,0x3> <2047,0x4>
<PDB,0x40F> System PT:<0,0x0> <1025,0x40C> <1026,0x40D> <1027,0x40E> <1028,0x410> <1029,0x411> <1030,0x412> <2047,0x413>
regs->eax = 4294967292 , u.u_error = 1
Process 1 is exiting
my:1's child 3 passed to 1#processProcess 3 is exiting
CTRL + 3rd button enables mouse | IPS: 40.800M | NUM | CAPS | SCRL | 10:0-1v

```



```
52 int kill(int pid, int signal)
53 {
54     int res;
55     __asm__ volatile ("int $0x80": "=a"(res): "a"(37), "b"(pid), "c"(signal) );
56     if ( res >= 0 )
57         return res;
58     return -1;
59 }
```